Statistical Methods in A.I.

Concept Deep Dive: Assignment - 1

March 21, 2026

Name: Harsh Kapoor
Roll No.: 2023112004
Email: harsh.kapoor@research.iiit.ac.in

GitHub link for all files: Assignment 1 Link

1 Conceptual Explanation

The Expectation-Maximization (EM) algorithm is used to estimate parameters in models with hidden variables, such as Gaussian Mixture Models (GMMs). In a GMM, each data point is assumed to come from one of several Gaussian distributions, but the identity of the generating component is unknown.

EM resolves this by alternating between two steps. In the E-step, it estimates how likely each data point belongs to each component (soft assignments). In the M-step, it updates the parameters of the Gaussians using these assignments.

This iterative refinement gradually improves the model fit to the data. However, the behavior of EM is governed by its convergence properties.

2 Mathematical Core

Given data $X = \{x_1, \ldots, x_N\}$, a GMM models:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2)$$

The log-likelihood is:

$$\ell(\theta) = \sum_i \log \sum_k \pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k^2)$$

E-step (Responsibilities):

$$\gamma_k(x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k^2)}{\sum_j \pi_j \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)}$$

M-step (Updates):

$$N_k = \sum_i \gamma_k(x_i), \quad \mu_k = \frac{1}{N_k} \sum_i \gamma_k(x_i) x_i, \quad \sigma_k^2 = \frac{1}{N_k} \sum_i \gamma_k(x_i)(x_i - \mu_k)^2, \quad \pi_k = \frac{N_k}{N}$$

Convergence Properties:

- **Monotonic Increase:** $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$ Each EM step improves the likelihood.
- **Local Maximum:** EM converges to a stationary point of $\ell(\theta)$, typically a local maximum due to non-convexity.
- **Sensitivity to Initialization:** Different initial parameters lead to different solutions.

3 Visual Element (1-D GMM Example)

Consider the dataset:

$$X = \{1, 2, 5, 8, 9\}$$

We initialize the model poorly:

$$\mu = (2, 4), \ \sigma = (1, 1), \ \pi = (0.5, 0.5)$$

Both Gaussian components initially lie near the left cluster, resulting in a poor fit to the data.

One EM Iteration (Representative Values)

E-step: The responsibilities for component 1 are approximately:

$$\gamma_1(1) \approx 0.99, \quad \gamma_1(2) \approx 0.97, \quad \gamma_1(5) \approx 0.15, \quad \gamma_1(8) \approx 0.01, \quad \gamma_1(9) \approx 0.00$$

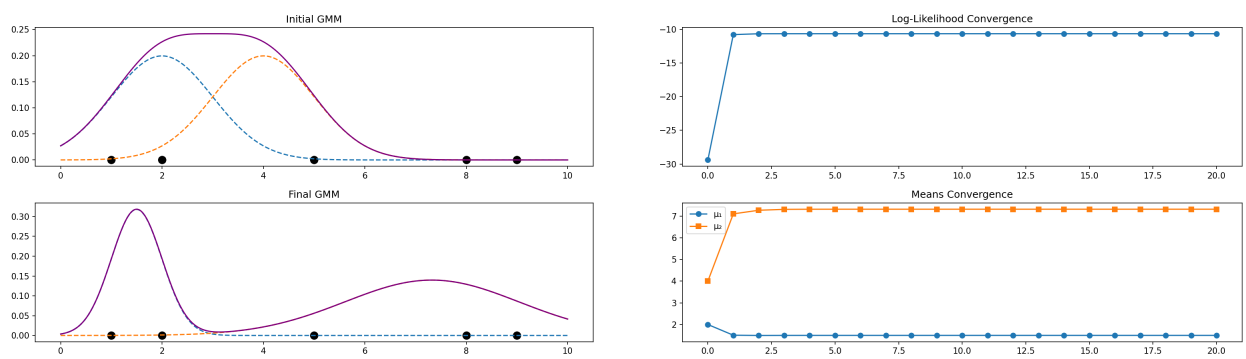
Thus, points $\{1, 2\}$ are strongly assigned to the first component, while $\{8, 9\}$ belong to the second. The point 5 is ambiguous and contributes to both.

M-step (effect): After one iteration, the updated parameters are approximately:

$$\mu_1 \approx 1.82, \quad \mu_2 \approx 6.68$$

This illustrates how EM performs *soft clustering* and gradually separates components.

Visualization



(a) Initial vs Final GMM

(b) Convergence behavior

Figure 1: EM evolution from poor initialization to converged model.

The left plot shows how both components initially overlap but eventually separate to match the data distribution. The right plot demonstrates that:

- The log-likelihood increases monotonically
- The means converge to stable values

Additionally, the animation (generated via code given in appendix) shows the step-by-step evolution of the Gaussian components, highlighting how EM iteratively refines parameters until convergence.

4 Connection to *K-Means*

EM for GMM has a strong connection to K-Means clustering. In GMM, each data point is assigned a *soft responsibility*:

$$\gamma_k(x_i) \in [0, 1]$$

which represents the probability of belonging to cluster k.

In the limit where the variance of each component shrinks ($\sigma_k^2 \to 0$), each Gaussian becomes highly concentrated around its mean. As a result, the responsibilities collapse to binary values:

$$\gamma_k(x_i) \to \{0, 1\}$$

leading to **hard assignments**. In this regime, the EM updates reduce to assigning each point to the nearest mean and recomputing cluster centers, which is exactly the K-Means algorithm.

More generally, EM can be viewed through the lens of variational inference. The algorithm maximizes a lower bound on the log-likelihood:

$$\ell(\theta) \geq \mathbb{E}_q[\log p(X, z \mid \theta)] - \mathbb{E}_q[\log q(z)]$$

by alternating between:

- optimizing $q(z)$ in the E-step (inference of latent variables),
- optimizing θ in the M-step (parameter learning).

This interpretation places EM within a broader class of approximate inference methods used in modern machine learning, where exact optimization is intractable due to latent variables.

5 Conclusion

EM provides a principled way to perform maximum likelihood estimation in models with latent variables. It guarantees monotonic improvement of likelihood and converges reliably, but only to local optima. This makes initialization an important practical consideration.

6 References

- C. Bishop, *Pattern Recognition and Machine Learning*
- K. Murphy, *Machine Learning: A Probabilistic Perspective*
- Course Lecture Notes
- Used AI tools for the code and animations used in this part.

Appendix:

```python
import numpy as np, matplotlib.pyplot as plt
from scipy.stats import norm
import matplotlib.animation as animation, os

os.makedirs("plots", exist_ok=True)

X=np.array([1,2,5,8,9])
mu,sigma,pi=np.array([2.,4.]),np.array([1.,1.]),np.array([.5,.5])
x=np.linspace(0,10,500)
mu_h,s_h,p_h,ll_h=[],[],[],[]

def ll(mu,sigma,pi):
    return np.sum(np.log(pi[0]*norm.pdf(X,mu[0],sigma[0])+pi[1]*norm.pdf(X,mu[1],sigma[1])))

def step(mu,sigma,pi):
    r=np.vstack([pi[k]*norm.pdf(X,mu[k],sigma[k]) for k in range(2)]).T
    r/=r.sum(axis=1,keepdims=True)
    Nk=r.sum(axis=0)
    mu=(r.T@X)/Nk
    sigma=np.sqrt(((r*(X[:,None]-mu)**2).sum(axis=0))/Nk)
    pi=Nk/len(X)
    return mu,sigma,pi

mu_h.append(mu.copy());s_h.append(sigma.copy());p_h.append(pi.copy());ll_h.append(ll(mu,sigma,pi))

for _ in range(20):
    mu,sigma,pi=step(mu,sigma,pi)
    mu_h.append(mu.copy());s_h.append(sigma.copy());p_h.append(pi.copy());ll_h.append(ll(mu,sigma,pi))

fig,ax=plt.subplots()
def upd(i):
    ax.clear()
    m,s,p=mu_h[i],s_h[i],p_h[i]
    ax.scatter(X,np.zeros_like(X),c='k')
    mix=p[0]*norm.pdf(x,m[0],s[0])+p[1]*norm.pdf(x,m[1],s[1])
    ax.plot(x,mix);ax.plot(x,p[0]*norm.pdf(x,m[0],s[0]),'--')
    ax.plot(x,p[1]*norm.pdf(x,m[1],s[1]),'--')
    ax.set_xlim(0,10);ax.set_ylim(0,0.5);ax.set_title(f"Iter {i}")
ani=animation.FuncAnimation(fig,upd,frames=len(mu_h),interval=800)
plt.show()

fig,(a,b)=plt.subplots(2,1,figsize=(10,6))
m0,s0,p0=mu_h[0],s_h[0],p_h[0]
a.scatter(X,np.zeros_like(X),c='k')
a.plot(x,p0[0]*norm.pdf(x,m0[0],s0[0]),'--')
a.plot(x,p0[1]*norm.pdf(x,m0[1],s0[1]),'--')
a.plot(x,p0[0]*norm.pdf(x,m0[0],s0[0])+p0[1]*norm.pdf(x,m0[1],s0[1]))
a.set_title("Initial")

mf,sf,pf=mu_h[-1],s_h[-1],p_h[-1]
b.scatter(X,np.zeros_like(X),c='k')
b.plot(x,pf[0]*norm.pdf(x,mf[0],sf[0]),'--')
b.plot(x,pf[1]*norm.pdf(x,mf[1],sf[1]),'--')
b.plot(x,pf[0]*norm.pdf(x,mf[0],sf[0])+pf[1]*norm.pdf(x,mf[1],sf[1]))
b.set_title("Final")

plt.tight_layout();plt.savefig("plots/gmm_initial_vs_final.png");plt.show()

fig,(a,b)=plt.subplots(2,1,figsize=(10,6))
it=np.arange(len(ll_h))
a.plot(it,ll_h);a.set_title("Log-Likelihood")
b.plot(it,[m[0] for m in mu_h]);b.plot(it,[m[1] for m in mu_h]);b.set_title("Means")
plt.tight_layout();plt.savefig("plots/gmm_convergence_metrics.png");plt.show()
```